common_idioms/mimic_methods.rb
```
class C
  def my_attribute=(value)
    @p = value
  end

  def my_attribute
    @p
  end
end

obj = C.new
obj.my_attribute = 'some value'
obj.my_attribute    # => "some value"
```

Writing obj.my_attribute = 'some value' is the same as writing obj.my_attribute=('some value'), but it looks cleaner.

What should we call disguised methods such as my_attribute and my_attribute=? Let's take a cue from zoology: an animal that disguises itself as another species is said to employ "mimicry." Following that pattern, a method call that disguises itself as something else, such as puts or obj.my_attribute=, can be called a *Mimic Method*.

Spell: Mimic Method

Mimic Methods are a very simple concept, but the more you look into Ruby, the more you find creative uses for them. For example, access modifiers such as private and protected are Mimic Methods, as are *Class Macros (117)* such as attr_reader. Popular libraries provide further examples. Here is one such example.

The Camping Example

The following snippet of code comes from an application written with the Camping web framework. It binds the /help URL to a specific controller action:

```
class Help < R '/help'
    def get
      # rendering for HTTP GET...
```

Class Help seems to inherit from a class named R. But what's that quirky little string right after R? You might assume that Ruby would simply refuse this syntax, until you realize that R is actually a Mimic Method that takes a string and returns an instance of Class. That is the class that Help actually inherits from. (If the notion of a method returning a class sounds strange to you, consider that Ruby classes are just objects, as you can read in Chapter 2, *Monday: The Object Model*, on page 11.)